

Tracing Program Printer

Kelompok B6

- Arya Nata Adidjaya (251524039)
- Fatur Hanafi (251524043)
- Muhammad Fawwaz Muzakki

1.Pseudocode

Program : Queue.c

```
// =====  
// MODUL QUEUE (Antrian Dokumen)  
// =====  
  
PROCEDURE buatQueue(Q : Pointer to Queue)  
    Q^.Head <- Nil  
    Q^.Tail <- Nil  
END PROCEDURE  
  
FUNCTION queueIsEmpty(Q : Queue) -> Boolean  
    RETURN (Q.Head == Nil) AND (Q.Tail == Nil)  
END FUNCTION  
  
FUNCTION Front(Q : Queue) -> String  
    RETURN Q.Head^.data  
END FUNCTION  
  
FUNCTION tampilkanQueue(Q : Queue) -> Integer  
    P <- Q.Head  
    i <- 0  
    WHILE P != Nil DO  
        PRINT P^.data  
        IF P^.next != Nil THEN  
            PRINT ", "  
        END IF  
        i <- i + 1  
        P <- P^.next  
    END WHILE  
    RETURN i  
END FUNCTION
```

```

PROCEDURE tambahQueue(Q : Pointer to Queue, X : String)
    ALOKASI Memori untuk P
    P^.data <- X
    P^.next <- Nil

    IF queueIsEmpty(Q^) THEN
        Q^.Head <- P
        Q^.Tail <- P
    ELSE
        Q^.Tail^.next <- P
        Q^.Tail <- P
    END IF
END PROCEDURE

FUNCTION hapusQueue(Q : Pointer to Queue) -> String
    IF queueIsEmpty(Q^) THEN
        RETURN NULL
    END IF

    temp <- Q^.Head
    buffer <- temp^.data
    Q^.Head <- temp^.next

    IF Q^.Head == Nil THEN
        Q^.Tail <- Nil
    END IF

    DEALOKASI temp
    RETURN buffer
END FUNCTION

PROCEDURE CetakAntrian(Q : Queue)
    IF queueIsEmpty(Q) THEN
        PRINT "Antrian kosong."
    ELSE
        PRINT "Kondisi antrian saat ini: [ "
        P <- Q.Head
        WHILE P != Nil DO
            PRINT P^.data, " "
            P <- P^.next
        END WHILE
        PRINT "]"
    END IF
END PROCEDURE

```

Program : Stack.c

```
// =====  
// MODUL STACK (Riwayat Cetak Dokumen)  
// =====  
  
PROCEDURE buatStack(s : Pointer to Stack)  
    s^.top <- NULL  
END PROCEDURE  
  
FUNCTION stackIsEmpty(s : Pointer to Stack) -> Boolean  
    RETURN (s^.top == NULL)  
END FUNCTION  
  
PROCEDURE tambahStack(s : Pointer to Stack, nilai :  
String)  
    ALOKASI Memori untuk newNode  
  
    IF newNode == NULL THEN  
        PRINT "Error: Alokasi memori gagal."  
        RETURN  
    END IF  
  
    newNode^.data <- nilai  
    newNode^.next <- s^.top  
    s^.top <- newNode  
END PROCEDURE  
  
FUNCTION hapusStack(s : Pointer to Stack) -> String  
    IF stackIsEmpty(s) THEN  
        RETURN ""  
    END IF  
  
    buffer <- s^.top^.data  
    temp <- s^.top  
    s^.top <- s^.top^.next  
  
    DEALOKASI temp  
    RETURN buffer  
END FUNCTION
```

```

PROCEDURE tampilkanStack(s : Pointer to Stack)
    IF stackIsEmpty(s) THEN
        PRINT " kosong "
        RETURN
    END IF

    curr <- s^.top
    WHILE curr != NULL DO
        PRINT curr^.data
        IF curr^.next != NULL THEN
            PRINT " -> "
        END IF
        curr <- curr^.next
    END WHILE
    PRINT (Baris Baru)
END PROCEDURE

```

Program : main.c

```

// =====
// MAIN PROGRAM (Sistem Printer)
// =====

// --- PROSEDUR-PROSEDUR PEMBANTU MAIN ---
PROCEDURE tambahDokumen(Q : Pointer to Queue)
    DEKLARASI nama : String

    PRINT "Masukkan nama dokumen: "
    INPUT nama
    tambahQueue(Q, nama)
    PRINT "Dokumen '", nama, "' ditambahkan ke antrian."
END PROCEDURE

PROCEDURE cetakDokumen(Q : Pointer to Queue, S : Pointer
to Stack)
    IF queueIsEmpty(Q) THEN
        PRINT "Antrian kosong, tidak ada dokumen untuk
dicetak."
        RETURN
    END IF

```

```

        END IF

        nama <- hapusQueue(Q)
        tambahStack(S, nama)
        PRINT "Mencetak dokumen: '", nama, "'"
    END PROCEDURE

PROCEDURE cancelCetak(S : Pointer to Stack)
    IF stackIsEmpty(S) THEN
        PRINT "Riwayat kosong, tidak ada yang bisa
dicancel."
        RETURN
    END IF

    nama <- hapusStack(S)
    PRINT "Cancel: dokumen '", nama, "' dihapus dari
riwayat."
END PROCEDURE

PROCEDURE tampilkanAntrianMain(Q : Queue)
    PRINT "Antrian : ["
    tampilkanQueue(Q)
    PRINT "]"
END PROCEDURE

PROCEDURE tampilkanRiwayatMain(S : Stack)
    PRINT "Riwayat : ["
    tampilkanStack(S)
    PRINT "]"
END PROCEDURE

// --- ALGORITMA UTAMA ---
PROGRAM UtamaSistemPrinter
DEKLARASI
    antrian : Queue
    riwayat : Stack
    pilihan : Integer

ALGORITMA
    buatQueue(Address Of antrian)
    buatStack(Address Of riwayat)
    pilihan <- 0

    WHILE (pilihan != 6) DO
        PRINT "\n=== Sistem Printer ==="

```

```

PRINT "1. Tambah Dokumen"
PRINT "2. Cetak Dokumen"
PRINT "3. Cancel Cetak Dokumen Terakhir"
PRINT "4. Tampilkan Antrian"
PRINT "5. Tampilkan Riwayat"
PRINT "6. Exit"
PRINT "Pilih operasi: "

INPUT pilihan

SWITCH (pilihan)
    CASE 1:
        tambahDokumen(Address Of antrian)
        BREAK
    CASE 2:
        cetakDokumen(Address Of antrian, Address
Of riwayat)
        BREAK
    CASE 3:
        cancelCetak(Address Of riwayat)
        BREAK
    CASE 4:
        tampilkanAntrianMain(antrian)
        BREAK
    CASE 5:
        tampilkanRiwayatMain(riwayat)
        BREAK
    CASE 6:
        PRINT "Keluar dari program."
        BREAK
    DEFAULT:
        PRINT "Pilihan tidak valid."
        BREAK
END SWITCH
END WHILE
END PROGRAM

```

2. Tracing Program

Langkah 1: Tambah Doc_A

Queue Q : [Doc_A]

Stack S : []

Langkah 2: Tambah Doc_B

Queue Q : [Doc_A → Doc_B]

Stack S : []

Langkah 3: Tambah Doc_C

Queue Q : [Doc_A → Doc_B → Doc_C]

Stack S : []

Langkah 4: Cetak

(Doc_A keluar antrian, masuk riwayat)

Queue Q : [Doc_B → Doc_C]

Stack S : [Doc_A]

Langkah 5: Cetak

(Doc_B keluar antrian, masuk riwayat)

Queue Q : [Doc_C]

Stack S : [Doc_B → Doc_A]

Langkah 6: Tambah Doc_D

Queue Q : [Doc_C → Doc_D]

Stack S : [Doc_B → Doc_A]

Langkah 7: Cancel

(Menghapus dokumen terakhir yang dicetak, yaitu Doc_B dari riwayat)

Queue Q : [Doc_C $\rightarrow$ Doc_D]

Stack S : [Doc_A]

Langkah 8: Cetak

(Doc_C keluar antrian, masuk riwayat)

Queue Q : [Doc_D]

Stack S : [Doc_C $\rightarrow$ Doc_A]

Langkah 9: Tambah Doc_E

Queue Q : [Doc_D $\rightarrow$ Doc_E]

Stack S : [Doc_C $\rightarrow$ Doc_A]

Langkah 10: Cetak

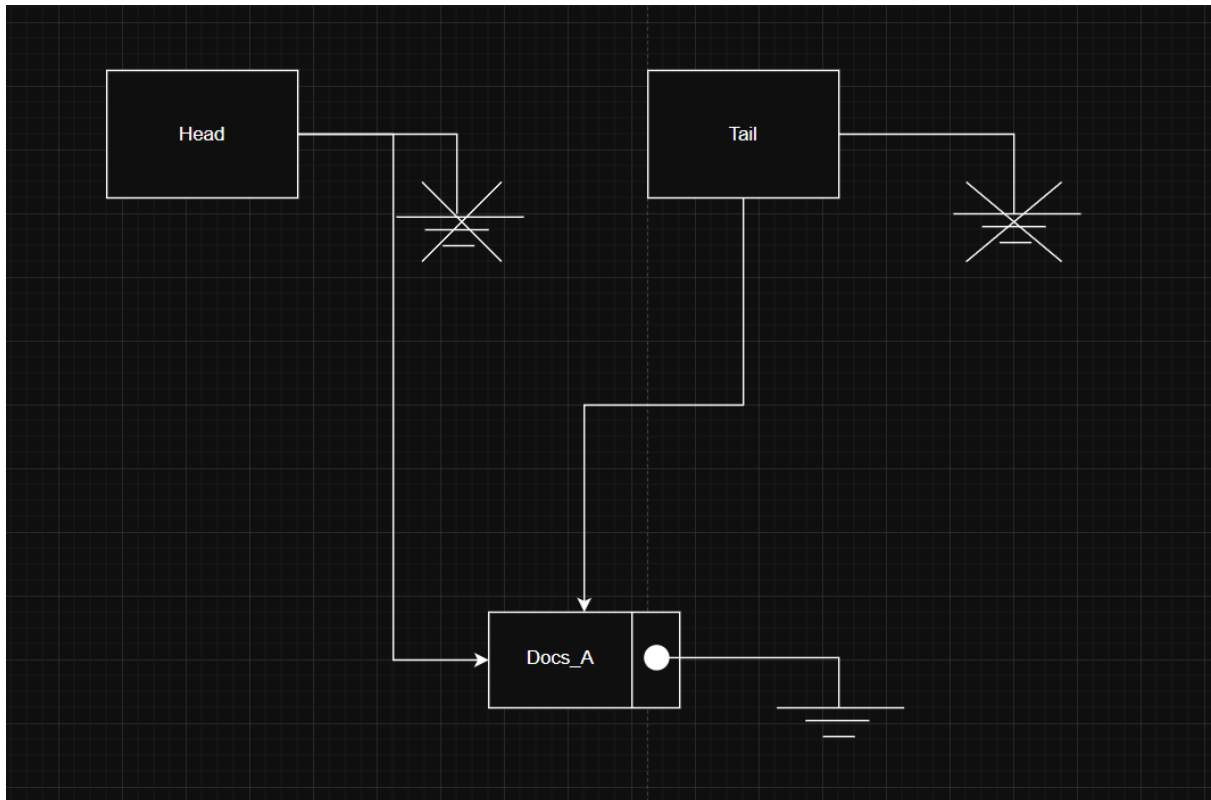
(Doc_D keluar antrian, masuk riwayat)

Queue Q : [Doc_E]

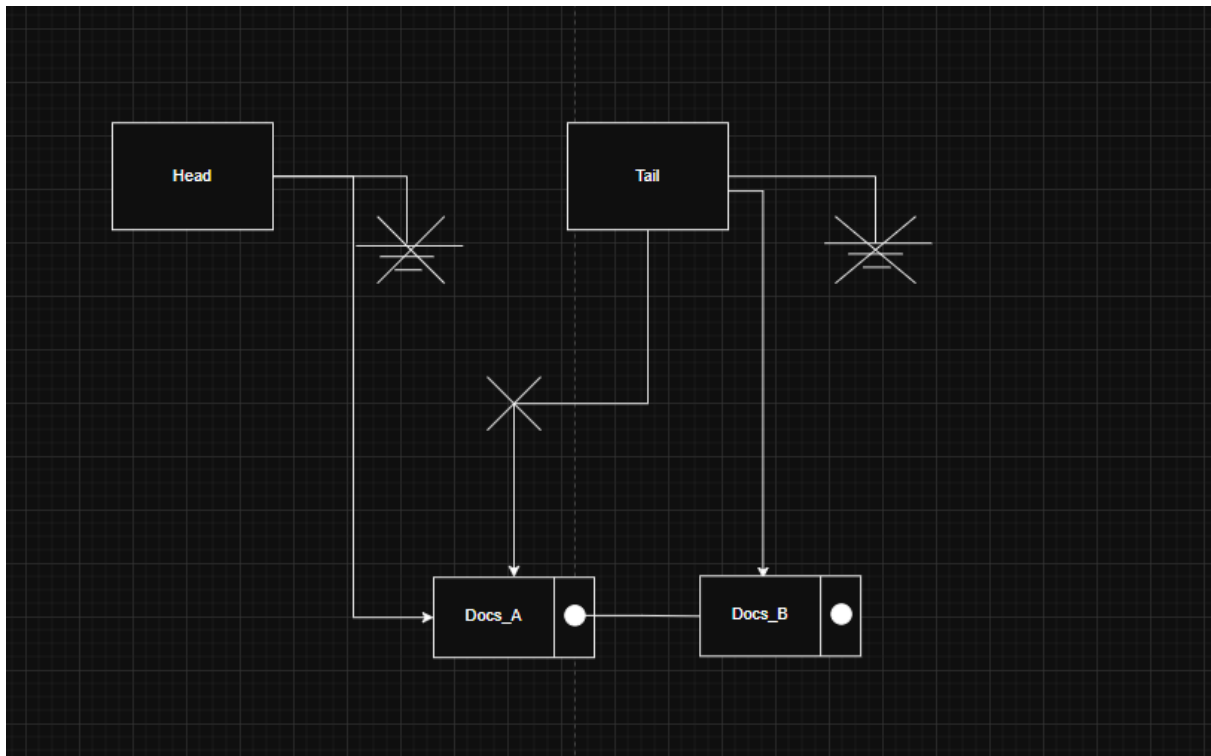
Stack S : [Doc_D $\rightarrow$ Doc_C $\rightarrow$ Doc_A]

3. Visualisasi

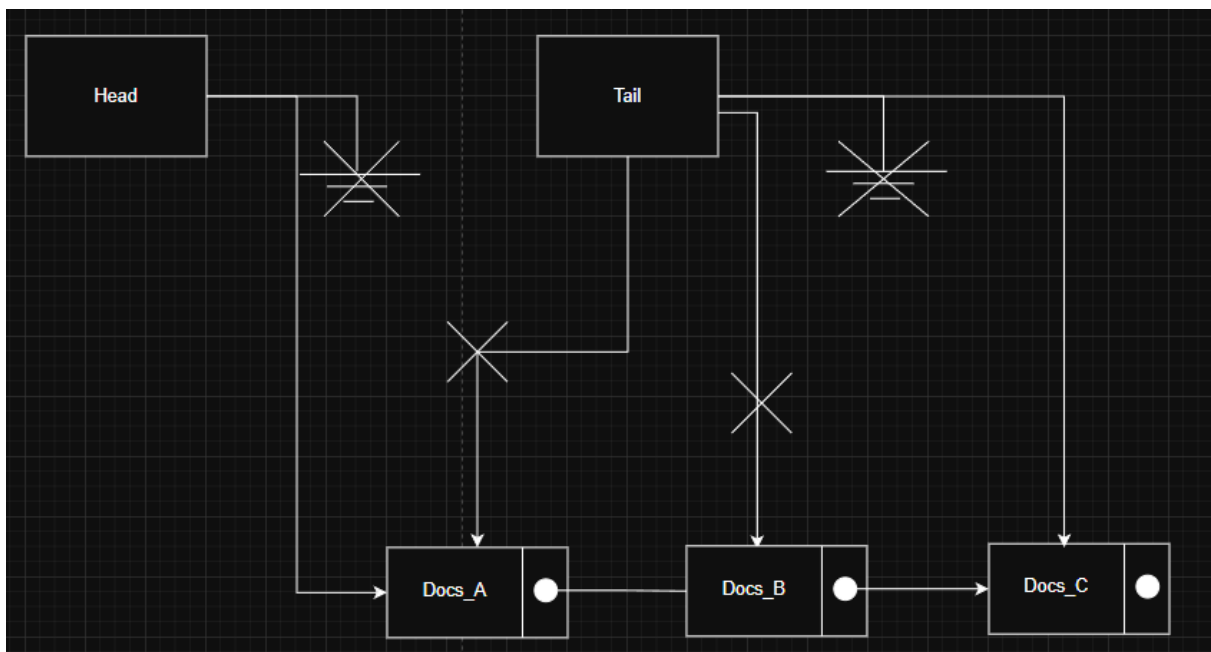
Langkah 1: Tambah Doc_A



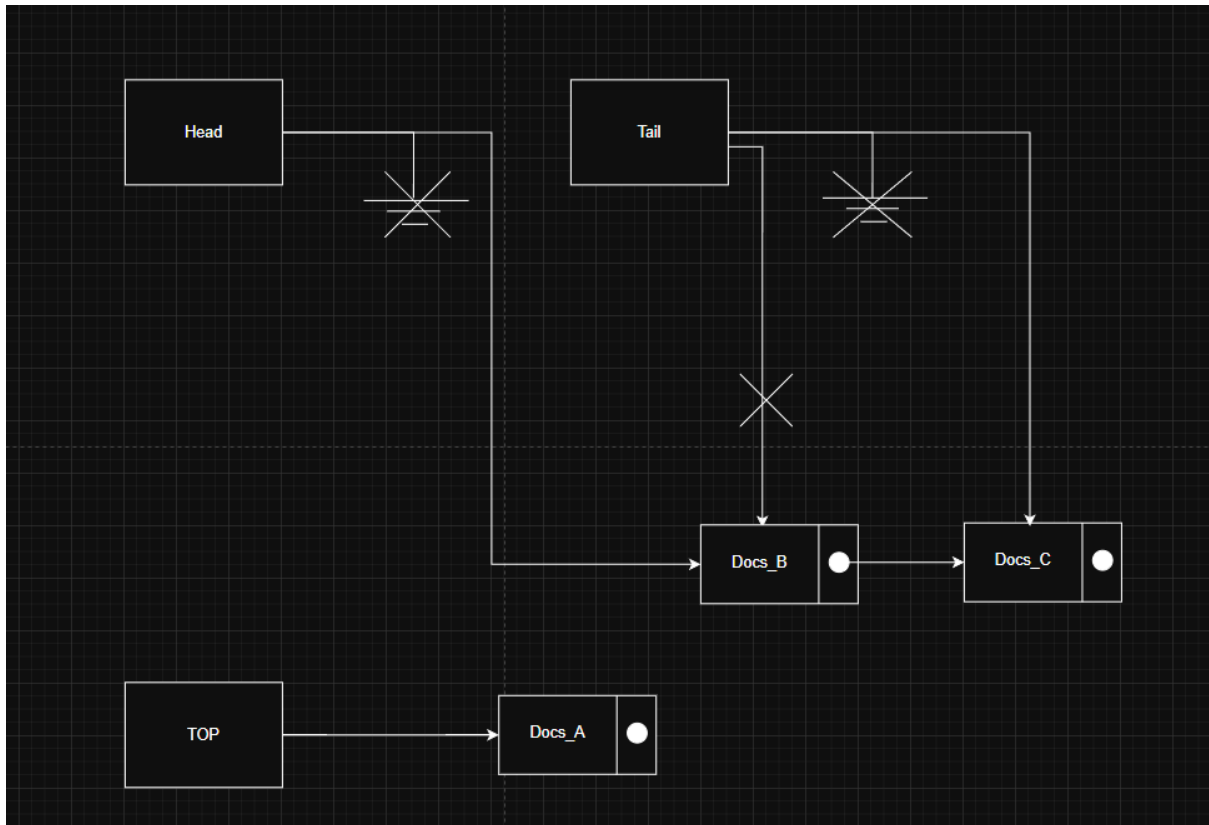
Langkah 2: Tambah Doc_B



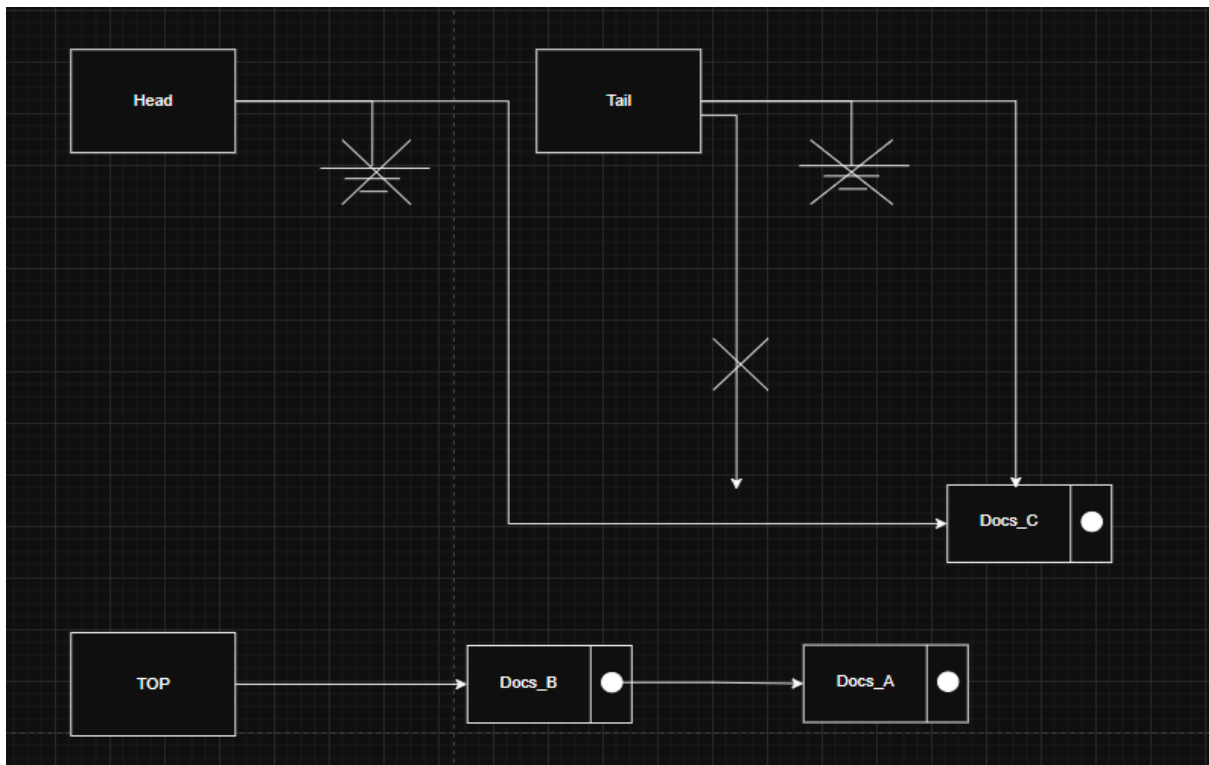
Tambahkan Doc_C



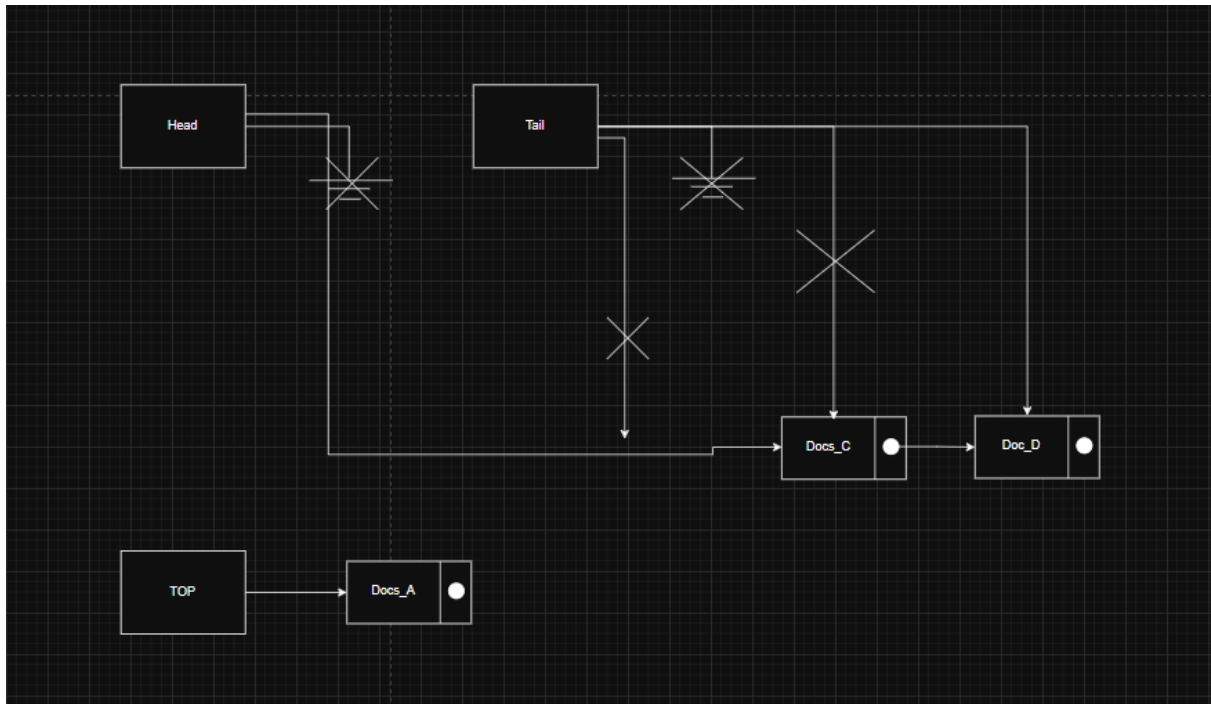
Langkah 4: Cetak



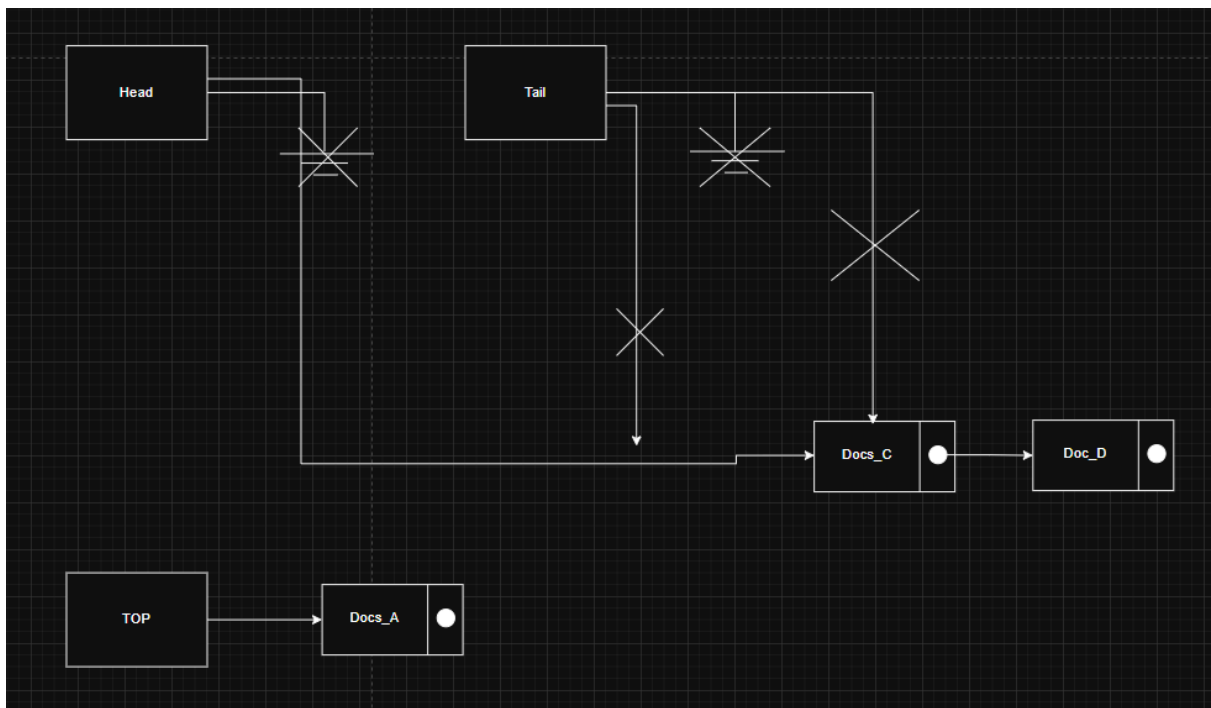
Langkah 5: Cetak



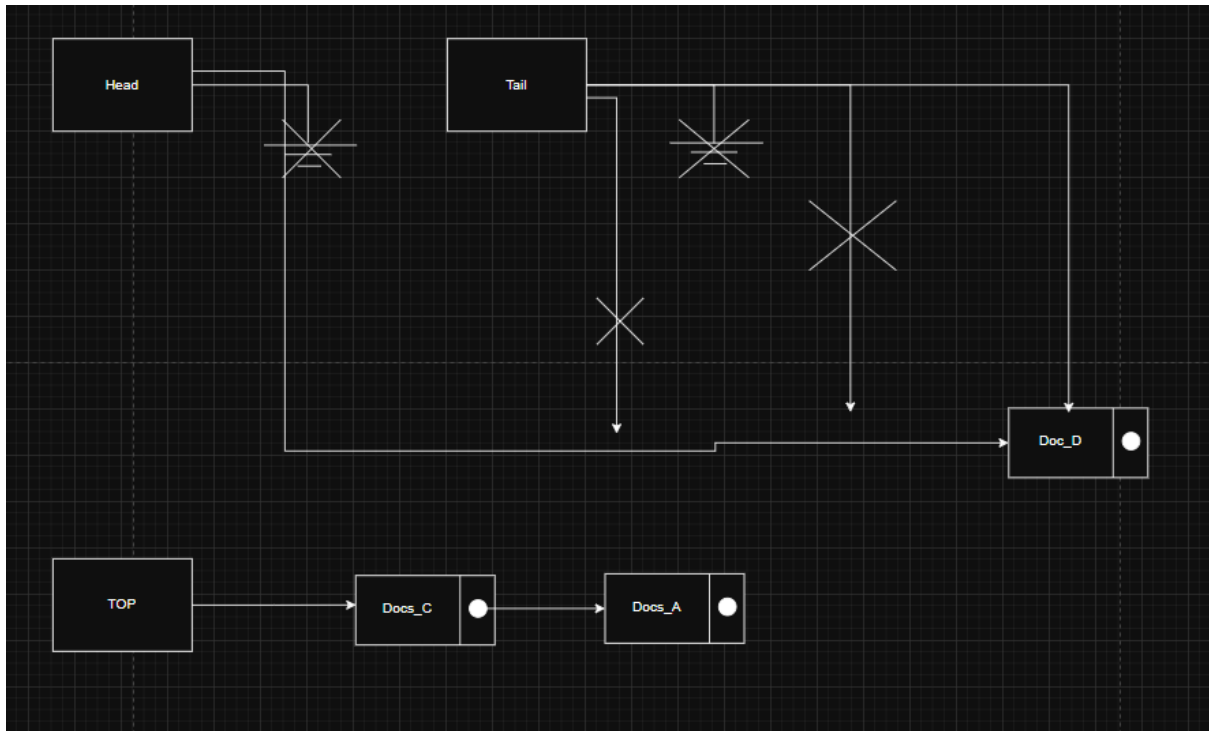
Langkah 6: Tambah Doc_D



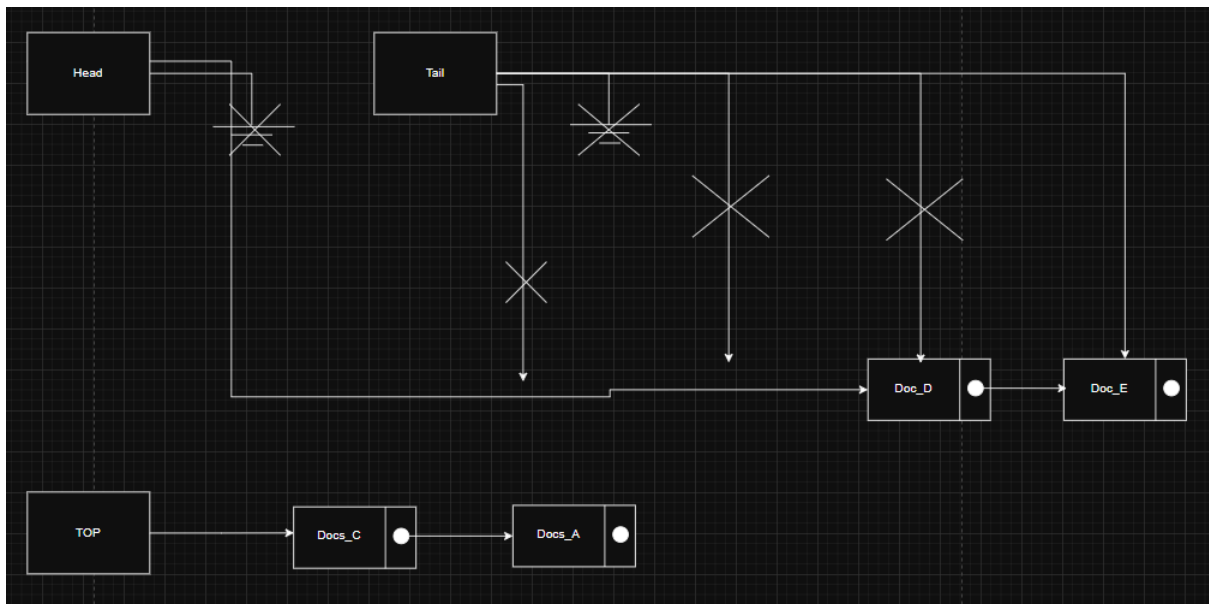
Langkah 7: Cancel



Langkah 8: Cetak



Langkah 9: Tambah Doc_E



Langkah 10: Cetak

